

Herald



Herald — commons_infra Module Guide (Operator / Developer)

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active
Status summary	Nano-detail per-module reference for commons_infra — Herald's on-demand container-infrastructure facade. Documents QuickstartBoot (the Up/Down/Status lifecycle that boots Postgres + Redis + OTel via the digital.vasic.containers submodule and opens the pgx pool + Redis client + task queue + dead-letter plane), the Pool/Queue/Redis/DeadLetterSink getters and the ErrNotBooted anti-bluff contract, the 19-method pgxTaskRepository backing the upstream PostgresTaskQueue (migrations 000009/000013/000014), and the WithTenantContext RLS bridge (which lives in commons_storage but is fed the pool this module opens). ANTI-BLUFF: every value below — container names, host ports, env-var names, method list — is transcribed from the real source under commons_infra/ + quickstart/docker-compose.quickstart.yml, NOT from prior prose.
Issues	(none specific to this guide) bump when (a) HRD-081 lands the compose --wait runtime split so the post-Up readiness poll can be removed, (b) the pherald doctor subcommand
Continuation	is implemented as a real consumer (today only Foundation tests call NewQuickstartBoot), and (c) the M2 Postgres-backed DeadLetterSink replaces the in-process MemoryDeadLetterSink.

Table of contents

- [§1. Overview](#)
- [§2. QuickstartBoot — construction and lifecycle](#)
- [§3. What Up actually boots \(the container stack — operator section\)](#)
- [§4. The client getters and the ErrNotBooted anti-bluff contract](#)
- [§5. The pgxTaskRepository \(19 methods\) + the queue seam](#)

- [§6. The governance dead-letter plane](#)
 - [§7. WithTenantContext — the RLS bridge](#)
 - [§8. How Foundation tests and pherald doctor consume it](#)
 - [§9. Usage examples](#)
 - [§10. References](#)
-

§1. Overview

`commons_infra` (Go package `infra`, module path `github.com/vasic-digital/herald/commons_infra`) is Herald's **on-demand container-infrastructure layer**. It is the thin Herald-side facade that brings Postgres + Redis + OTel up on-demand for tests and tooling, opens the live client handles against them, and tears it all down — so a test entry point never assumes the operator pre-started podman/docker by hand.

Per the package doc at the top of `boot.go`, the module exists to satisfy **Universal Constitution §11.4.76** (the containers-submodule mandate) and **Herald V3 spec §44**: Foundation tests + the `pherald doctor` subcommand **MUST** boot infrastructure via the canonical `digital.vasic.containers` submodule's `pkg/compose` orchestrator — **never** via ad-hoc `docker compose` shellouts. `commons_infra` wraps that orchestrator with Herald-flavor defaults (project name, compose-file path, readiness polling) and adds the client-opening + dead-letter wiring on top.

The module is anti-bluff by construction (§107 / §11.4.5): calling `Up` is not enough — the getters return `ErrNotBooted` until a client is actually populated, and the package doc explicitly states that a test which calls `Up` but never verifies the services are reachable is **still a bluff**.

§2. QuickstartBoot — construction and lifecycle

`QuickstartBoot` (in `boot.go`) wraps a `compose.ComposeOrchestrator` + `compose.ComposeProject` with Herald defaults and holds the opened clients.

§2.1 Config and NewQuickstartBoot

```
type Config struct {
    ComposeFile string          // empty → auto-resolve <repo-root>/quickstart/docker-compose.quickstart.yml
    ProjectName string          // empty → DefaultProjectName ("herald-quickstart")
    Services    []string                   // empty → all services; otherwise a subset to boot + open clients for
    Logger      logging.Logger             // nil → NopLogger
}
```

```
func NewQuickstartBoot(cfg Config) (*QuickstartBoot, error)
```

`const DefaultProjectName = "herald-quickstart" — matches the project name the §26.5 compose file declares.`

`NewQuickstartBoot` resolves the compose file (via `findQuickstartCompose`, which walks **up to 16 parent directories** from `os.Getwd` looking for `quickstart/docker-`

`compose.quickstart.yml`), defaults the project name, sets the orchestrator working directory to the compose file's directory (so relative build-context / env-file paths inside the compose resolve), and constructs a default auto-detected orchestrator via `compose.NewDefaultOrchestrator`. It returns an **error if no compose runtime (docker / podman / podman-compose) is available on the host** — the §11.4.76 invariant assumes one is installed, because the boot is the *first* step of the test entry point, not "operator already ran podman manually".

§2.2 Up(ctx) — bring up + open clients

```
func (b *QuickstartBoot) Up(ctx context.Context) error
```

Up runs `orch.Up` with `WithUpDetach(true) + WithUpTimeout(120)`, then opens clients for the requested services. Load-bearing behaviours, all transcribed from the source:

- **No --wait.** `compose.WithWait(true)` is intentionally NOT passed, because `podman-compose` (the canonical Herald-dev runtime) does not recognise the `--wait` flag (see HRD-081). Callers must therefore follow Up with their own readiness check (`Status()` loop or a service-specific probe) before trusting the infra.
- **Idempotency guard.** If `b.pool != nil || b.redis != nil`, Up short-circuits — re-opening would leak pgx connections + goroutines and race the Redis healthcheck.
- **Selective opens.** `serviceRequested("postgres") / serviceRequested("redis")` gate which clients open; an empty `Config.Services` is the "all services" sentinel (every name returns true).
- **Postgres open + readiness poll.** When Postgres is requested, Up builds a config via `storage.ConfigForHerald(...)` and **retries** `storage.Open` **for up to 30s** (500ms poll interval) — `podman-compose` can report the container "Up" while Postgres is still mid-`pg_ctl start (SQLSTATE 57P03)`. On success it runs `storage.RunMigrations(ctx, pool)` so the schema is live before any test touches the pool.
- **Queue bind.** After the pool opens, Up binds the upstream `digital.vasic.background.PostgresTaskQueue` to the pool via Herald's local `pgxTaskRepository` (see §5).
- **Dead-letter plane.** Alongside the pool/queue, Up stands up the full closed-loop governance dead-letter plane (see §6).
- **Redis open + healthcheck.** When Redis is requested, Up opens the client and calls `rc.HealthCheck(ctx)`. On healthcheck failure it **rolls back the pool/queue/dead-letter plane** it just opened (so a partial boot doesn't leave `b.pool` dangling and let a retry's idempotency guard short-circuit on a half-booted lifecycle).

§2.3 Down(ctx) — graceful teardown

```
func (b *QuickstartBoot) Down(ctx context.Context) error
```

Down closes **clients FIRST, then compose** — the order matters: it closes `b.pool`, then `b.redis`, nils the `b.queue` field (the `PostgresTaskQueue` borrows the now-closed pool — clearing the field stops a stale dequeue 500-ing, the §107 PASS-bluff this prevents), shuts down the dead-letter plane, and only then runs `orch.Down`. Closing clients before compose-down means pgx isn't flushing against an already-killed Postgres. Down is idempotent.

§2.4 Status(ctx)

```
func (b *QuickstartBoot) Status(ctx context.Context)
    ([]compose.ServiceStatus, error)
```

Returns per-service status as reported by `compose ps`. This is the readiness-poll primitive callers use after Up (since `--wait` is unavailable, §2.2), and the probe `herald doctor` is intended to surface.

§3. What Up actually boots (the container stack — operator section)

Anti-bluff correction. The boot brings up the services declared in `quickstart/docker-compose.quickstart.yml`. The **real** container names and host ports below are transcribed verbatim from that file as of this revision — they are in the **24XXX** reserved range (spec §9.4), with `herald--`-prefixed container names. (Earlier prose elsewhere referencing a "70XXX" range or a bare `herald` prefix does not match the committed compose; trust the table below.)

Service	Container name	Host port → container port	Purpose
Postgres	herald-postgres	24100:5432	Main DB (multi-tenant, RLS-enforced).
Redis	herald-redis	24200:6379	In-memory cache; <code>--requirepass</code> enabled.
OTel Collector	herald-otel	24417:4317 (OTLP gRPC), 24418:4318 (OTLP HTTP)	Telemetry collection (<code>otel/opentelemetry-collector-contrib:0.115.0</code>).
pherald (app)	herald-pherald	24091:24091 (webhook ingress), 24090:24090 (admin: /livez, /readyz, /metrics, /admin/version)	The flavor binary itself (booted by the full compose; not opened as a client by this module).

`commons_infra` itself only opens **client handles** for Postgres and Redis (the OTel collector is consumed by the running binaries via `OTEL_EXPORTER_OTLP_ENDPOINT`, not by this module). The Postgres/Redis connection defaults Up uses (host `127.0.0.1`, ports `24100/24200`) match the host-side compose mappings above and are overridable via env vars (§3.1) without code changes.

§3.1 Env-var overrides (envOr / envOrInt)

Up reads these via the `envOr` (string) / `envOrInt` (int) helpers; each falls back to the Herald development default when the var is unset or empty:

Env var	Default	Used for
HERALD_DB_HOST	127.0.0.1	Postgres host
HERALD_DB_PORT	24100	Postgres host port
HERALD_DB_USER	herald	Postgres user
HERALD_DB_PASSWORD	herald_dev	Postgres password
HERALD_DB_NAME	herald	Postgres database

Env var	Default	Used for
HERALD_REDIS_ADDR	127.0.0.1:24200	Redis address
HERALD_REDIS_PASSWORD	" " (empty)	Redis password (compose requires the operator supply one)
HERALD_REDIS_DB	0	Redis logical DB index

These are the canonical names aligned with the compose file (the earlier `HERALD_PG_PASSWORD` chained drift was removed in HRD-010 Task 4). Per Herald's credential resolution order, exported shell vars win over `.env`.

§4. The client getters and the `ErrNotBooted` anti-bluff contract

`clients.go` exposes the handles `Up` populated. Every getter returns `ErrNotBooted` when its field is `nil` — and the package doc states plainly: a caller that gets `nil` + `ErrNotBooted` and `PASSes` without asserting **is a bluff**.

```
var ErrNotBooted =
    errors.New("commons_infra: QuickstartBoot.Up() not called
    or failed; clients unavailable")

func (b *QuickstartBoot) Pool() (database.Database, error)    //
    driver-agnostic pgx-backed pool
func (b *QuickstartBoot) Queue() (TaskQueue, error)          //
    = digital.vasic.background.TaskQueue
func (b *QuickstartBoot) Redis() (*redis.Client, error)       //
    *digital.vasic.cache/pkg/redis.Client
func (b *QuickstartBoot) DeadLetterSink() (DeadLetterSink,
    error) // durable dead-letter audit sink
```

- `Pool` returns `digital.vasic.database/pkg/database.Database` — the **driver-agnostic interface**, not a raw `pgxpool.Pool`, so it composes with the Helix-stack abstraction.
- `Queue` returns `TaskQueue` (a type alias for `digital.vasic.background.TaskQueue`).
- `Redis` returns the **concrete** `*redis.Client` struct (the upstream's surface is a struct, not an interface).
- `DeadLetterSink` returns `ErrNotBooted` when the boot opened **no Postgres pool** — the dead-letter plane is stood up only alongside the pool/queue (§6).

Callers **MUST** check the error and abort; `clients_test.go` pins this contract via `errors.Is(err, ErrNotBooted)`.

§5. The `pgxTaskRepository` (19 methods) + the queue seam

The upstream `digital.vasic.background.PostgresTaskQueue` requires a `background.TaskRepository` implementation but does **not** ship one — the schema (columns, indexes) is consumer-specific. Per §11.4.74 (extend-don't-reimplement), Herald reuses the upstream queue's `Enqueue/Dequeue/Peek/Requeue/MoveToDeadLetter/`

GetPendingCount/GetRunningCount/GetQueueDepth *logic* and owns only the thin SQL-binding repository underneath it: `pgxTaskRepository` in `task_repository.go`.

It holds the universal `db.Database` interface (not the raw pool) plus an **optional** `constitution.EventEmitter`. Two constructors:
`newPgxTaskRepository(database)` (no emitter) and
`newPgxTaskRepositoryWithEmitter(database, emitter)` (publishes `.queue.dead_letter` on dead-letter moves — see §6). The emitter is nil-tolerant: the move still happens, the event is simply not published.

The repository implements **19** methods against the `background_tasks` + `background_task_events` + `task_resource_snapshots` + `dead_letter_tasks` tables:

#	Method	Notes
1	Create	Insert; columns map 1:1 to migration 000009 <code>background_tasks</code> . Called by <code>Enqueue</code> .
2	Dequeue	Resource-filtered claim by <code>workerID</code> + CPU/memory caps.
3	LogEvent	Appends a <code>background_task_events</code> row.
4	GetByID	Single-task fetch.
5	Update	Full-row update.
6	Delete	Soft-delete (preserves FK children).
7	UpdateStatus	Status transition.
8	UpdateProgress	Progress fraction + message.
9	UpdateHeartbeat	Liveness stamp.
10	SaveCheckpoint	Resumable-task checkpoint blob.
11	GetByStatus	Paged by status (<code>limit/offset</code>).
12	GetPendingTasks	The dispatch read.
13	CountByStatus	<code>map[TaskStatus]int64</code> aggregate.
14	GetTaskHistory	<code>task_execution_history</code> rows for a task.
15	GetStaleTasks	Heartbeat-threshold stuck-detector read.
16	GetByWorkerID	All tasks claimed by a worker.
17	SaveResourceSnapshot	Append into <code>task_resource_snapshots</code> (migration 000013 , HRD-089); UUIDv7 PK so samples sort chronologically.
18	GetResourceSnapshots	Reverse-chronological time-series read; <code>limit <= 0</code> rejected.
19	MoveToDeadLetter	Atomic move into <code>dead_letter_tasks</code> (migration 000014 , HRD-090): full JSONB snapshot INSERT + mark source row terminal <code>status='dead_letter'</code> . Mark-terminal (not hard-DELETE) so the FK + audit children survive. Publishes <code>.queue.dead_letter</code> best-effort when an emitter is wired.

Migrations **000013** (`task_resource_snapshots`) and **000014** (`dead_letter_tasks`) both deliberately carry **no** `tenant_id` / **RLS** — they are dispatcher-internal queue/monitoring infrastructure; tenant context lives in the parent task's JSON payload, enforced one level up by Herald service code. Both FK to `background_tasks(id)` ON DELETE CASCADE.

`queue.go` is the alias seam: `Task = models.BackgroundTask, TaskPriority = models.TaskPriority, ResourceRequirements = bg.ResourceRequirements, TaskQueue = bg.TaskQueue`. Any upstream drift trips a compile error here — that's the §11.4.74 in-place-extension contract.

§6. The governance dead-letter plane

When Up opens the Postgres pool, it also stands up the **full closed-loop** dead-letter plane (HRD-147) — previously the boot used a nil emitter, which is an "emit-into-the-void" §107 bluff. The plane has three pieces, wired in `boot.go` and `dead_letter_subscriber.go`:

1. `constitution.NewMemoryBus` — the in-process `EventBus`.
2. `DeadLetterSubscriber(StartDeadLetterSubscriber(bus, sink))` — owns one drain goroutine that filters `.queue.dead_letter` events (`constitution.EventNamespace + "." + constitution.ClassQueueDeadLetter`), decodes each, and records a `DeadLetterRecord` into the sink. Exposes `Stats()` (`recorded, decodeErrors int`) and `Close()`.
3. `constitution.Emitter(source digital.vasic.herald/commons_infra)` handed to the repository via `newPgxTaskRepositoryWithEmitter`, so `MoveToDeadLetter` publishes to a **real consumer**, not the void.

The sink is `DeadLetterSink(RecordDeadLetter + List, append-only)`. The shipped backend is `MemoryDeadLetterSink(NewMemoryDeadLetterSink)` — slice-backed, mutex-guarded (safe under `-race`), with a convenience `Len()`. A `DeadLetterRecord` carries `EventID`, `TaskID`, `Reason`, `FailureCount`, `Severity`, `RuleID`, `RecordedAt`. The M2 swap is a Postgres-backed sink behind the same interface — the subscriber is unchanged when it lands.

Guard rails: if either the subscriber or the emitter can't be constructed, Up falls back to the nil-emitter repository (the move still happens; the audit event is simply not published) rather than failing the whole boot — a dead-letter audit gap must not take down the queue. `shutdownDeadLetterPlane()` (called by `Down` and on partial-boot rollback) is idempotent: it `Close()`s the subscriber (joins the goroutine), closes the bus, and clears all three fields.

§7. WithTenantContext — the RLS bridge

Where it lives. `WithTenantContext` is defined in `commons_storage/postgres.go`, not in `commons_infra`. It is documented here because the pool `commons_infra.Up` opens (and exposes via `Pool()`) is exactly the `database.Database` you pass into it — `commons_infra` is the layer that produces the RLS-ready pool, `commons_storage` is the layer that scopes it per tenant.

```
func WithTenantContext(ctx context.Context, database
    db.Database, tenantID uuid.UUID, fn func(tx db.Tx)
    error) error
```

`WithTenantContext` runs `fn` inside a single transaction with two `SET LOCAL`s pre-applied (per spec §16 + §44.6):

1. `SET LOCAL ROLE herald_app` — the load-bearing step. The bootstrap `POSTGRES_USER` (the `herald` role the pool authenticated as) is typically `SUPERUSER`, which **bypasses RLS regardless of FORCE ROW LEVEL SECURITY**. `herald_app` is `NOBYPASSRLS` (migration 000001) and holds `CRUD` grants on all multi-tenant tables (migration 000008). Without this downgrade, tenant-isolation tests would PASS while production was wide open — the exact §107 E14 bluff discovered 2026-05-20.
2. `SET LOCAL app.tenant_id = '<uuid>'` — the GUC the RLS policies read. Inlined (not parameterised) because `SET LOCAL` rejects bind parameters; `uuid.UUID.String()` constrains the format so injection isn't possible.

It commits if `fn` returns `nil`, rolls back otherwise (deferred rollback guard), and surfaces a commit error. `SET LOCAL` is transaction-scoped, so the role + tenant context cleanly evaporate at commit/rollback. The round-trip proof lives in `commons_storage/storage_integration_test.go` (the E14 RLS test, which runs against the very pool `commons_infra` boots).

§8. How Foundation tests and pherald doctor consume it

Foundation tests (live today). The canonical consumer pattern is a `TestMain` that boots once for the whole package, per the package doc:

```
func TestMain(m *testing.M) {
    ctx := context.Background()
    boot, err := infra.NewQuickstartBoot(infra.Config{})
    if err != nil { log.Fatal(err) }
    if err := boot.Up(ctx); err != nil { log.Fatal(err) }
    code := m.Run()
    _ = boot.Down(context.Background())
    os.Exit(code)
}
```

Integration tests across the workspace consume it this way — e.g. `commons_infra/boot_integration_test.go`, `clients_integration_test.go`, `task_repository_integration_test.go`, `dead_letter_integration_test.go` (+ its stress/chaos sibling), and `commons_storage/storage_integration_test.go` (the RLS proof of §7). They follow `Up` with a real reachability check (`Status()` / a `TCP/SELECT 1` probe / the `WaitHealthy`-style poll) before trusting the `infra` — the §11.4.76 anti-bluff requirement.

pherald doctor (documented-intended). The package doc and Herald's `CLAUDE.md` describe `pherald doctor` as the operator-facing consumer that brings Postgres + Redis + OTEL up on-demand and reports per-service health via `Status()`. As of this revision the wiring lives in the boot facade; the dedicated `doctor` subcommand is intended work, so treat the Foundation-test path above as the load-bearing live consumer and `doctor` as the operator entry point it backs.

§9. Usage examples

§9.1 Boot everything, use the pool under a tenant scope

```
boot, err := infra.NewQuickstartBoot(infra.Config{})
if err != nil { /* no compose runtime – install podman/docker
               */ }
if err := boot.Up(ctx); err != nil { /* boot failed */ }
defer boot.Down(context.Background())

pool, err := boot.Pool()
if err != nil
    { /* errors.Is(err, infra.ErrNotBooted) – abort, do NOT
      PASS */ }

// Tenant-scoped write under RLS (WithTenantContext lives in
// commons_storage):
err = storage.WithTenantContext(ctx, pool, tenantID, func(tx
    db.Tx) error {
    _, e := tx.Exec(ctx, "INSERT INTO ... VALUES ($1)", val)
    return e
})
```

§9.2 Boot only Redis (subset)

```
boot, _ := infra.NewQuickstartBoot(infra.Config{Services:
    []string{"redis"}})
_ = boot.Up(ctx)
rc, err :=
    boot.Redis() // Pool()/Queue() return ErrNotBooted –
    Postgres wasn't requested
```

§9.3 Inspect the dead-letter audit trail

```
sink, err :=
    boot.DeadLetterSink() // ErrNotBooted if no Postgres pool
    was opened
if err == nil {
    records, _ := sink.List(ctx) // every
    recorded .queue.dead_letter occurrence, oldest-first
}
```

§10. References

- Source: commons_infra/boot.go, clients.go, queue.go, task_repository.go, dead_letter_subscriber.go.

- Containers submodule: `digital.vasic.containers` (`pkg/compose` orchestrator + `pkg/logging`) — runtime auto-detection + on-demand boot/lifecycle/health. Imported via `replace` from `submodules//containers/`.
- Compose file: `quickstart/docker-compose.quickstart.yml` (the §26.5 5-minute operator quickstart — container names + the 24XXX host ports in §3).
- Storage layer: `commons_storage` — `ConfigForHerald`, `Open`, `RunMigrations`, and `WithTenantContext` (`commons_storage/postgres.go`, §7).
- Migrations: `commons_storage/migrations/000009_background_tasks`, `000013_task_resource_snapshots`, `000014_dead_letter_tasks`.
- Spec: V4 §26.5 (operator quickstart), §44 (Foundation implementation contract), §16 + §44.6 (RLS / tenant context), §9.4 (reserved 24XXX host-port range).
- Constitution: Universal §11.4.76 (containers-submodule mandate), §11.4.74 (extend-don't-reimplement), §107 / §11.4.5 (anti-bluff captured-evidence).
- Related guides: `docs/guides/COMMONS_WATCH.md`, `docs/guides/COMMONS_WORKABLE.md`, `docs/guides/OPERATOR_CREDENTIALS.md`.

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on. All behavioural claims — the `Up/Down/Status` lifecycle, the `ErrNotBooted` getter contract, the 19 `pgxTaskRepository` methods, the dead-letter plane wiring, and `WithTenantContext` — are grounded in the cited source files read on 2026-05-31. Container names + host ports (`herald-postgres` 24100, `herald-redis` 24200, `herald-otel` 24417/24418, `herald-pherald` 24090/24091) are transcribed verbatim from `quickstart/docker-compose.quickstart.yml`.

Verified 2026-05-31: internal doc — no external online sources. Re-verify on a `digital.vasic.containers/digital.vasic.background` API change, a compose-file port/container-name change, or when the `pherald doctor` subcommand and the M2 Postgres-backed `DeadLetterSink` land.